

DOCUMENTATION

Utility Asset Registry

1. Overview
2. Architecture
3. Setup
4. Accounts & roles
5. Ingestion CLI
6. Cleaning & validation
7. Web API
8. Reports
9. Security & limits
10. Database
11. Output files
12. Testing
13. Project layout
14. Quick reference

Utility Asset Registry

Backend for a state electricity distribution utility. It cleans daily handheld GPS survey exports, stores trusted asset records in PostgreSQL, and serves them over HTTP to authorised staff and the utility web map.

[Python](#)[FastAPI](#)[PostgreSQL](#)[CLI + REST API](#)[Version 1.0.0](#)

1. Overview

Field crews export a day's work as a CSV. This application replaces manual re-typing into a spreadsheet with automated cleaning, permanent storage, access control, and queryable network APIs.

Part A — Ingestion tool

Night-shift operators run one terminal command. Bad rows are rejected with reasons; good rows are stored.

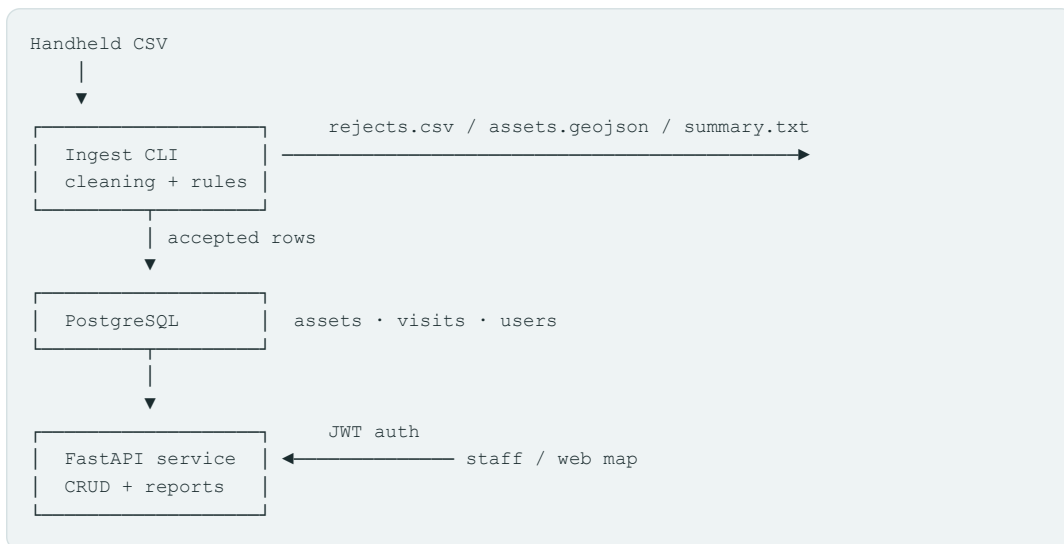
Part B — Web service

Staff and the web map call authenticated HTTP endpoints. Interactive docs live at [/docs](#).

Capabilities

- Load a day's CSV with a single command
- Standardise descriptions, coordinates, types, and surveyor names
- Never drop a bad row silently — write it to a rejects file with a reason
- Refuse duplicate `asset_id` values already in the database or the same file
- Store assets and visit history in PostgreSQL
- Search, filter, and page assets over the network
- Role-based access: surveyor vs administrator
- Cached summary reports for the web map
- Request logging, rate limiting, and CORS for a configured map origin

2. Architecture



Domain logic (cleaning, validation, distance) is shared by the CLI and the API so the same rules apply whether data arrives from a file or over the network.

3. Setup

Requirements

- Python 3.11+
- PostgreSQL Server (local or enterprise)

Python environment

```
copy .env.example .env
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
python scripts/setup_local_database.py
```

```
alembic upgrade head
python main.py
```

PostgreSQL / pgAdmin (this machine)

Setting	Value
Host	localhost
Port	5433 (local PostgreSQL 18)
Database	asset_registry
App user	asset_registry / assetreg123

If the database password contains special characters such as @, URL-encode them in `DATABASE_URL` (for example @ → %40).

Create the role and databases with `python scripts/setup_local_database.py` or by running `scripts/init_local_database.sql` in pgAdmin.

4. Accounts & roles

Role	Username	Password	Permissions
Administrator	admin	Admin@123	Read, add, update, delete, bulk upload, create users
Surveyor	surveyor	Surveyor@123	Read, add, update only (delete refused)

Sign in via `POST /auth/login`. The response includes an `access_token`. Send it on later requests as:

```
Authorization: Bearer <access_token>
```

The bootstrap admin is created (or password-synced) from `BOOTSTRAP_ADMIN_USERNAME` / `BOOTSTRAP_ADMIN_PASSWORD` in `.env` when the API starts.

5. Ingestion CLI

For operators who do not need to know Python:

```
python -m asset_registry.ingest data/survey_export.csv
```

Options

```
python -m asset_registry.ingest --help
python -m asset_registry.ingest data/survey_export.csv ^
  --rejects outputs/rejects.csv ^
  --map outputs/assets.geojson ^
  --report outputs/summary.txt
python -m asset_registry.ingest data/survey_export.csv --strict
```

Option	Meaning
csv_path	Path to the handheld export (required)
--rejects	Rejects file path (default <code>outputs/rejects.csv</code>)
--map	GeoJSON map file (default <code>outputs/assets.geojson</code>)
--report	Plain-text summary (default <code>outputs/summary.txt</code>)
--strict	Abort the whole run on the first rejected row

Behaviour

- Confirms every expected column is present before processing
- Accepts valid new assets into PostgreSQL
- Rejects invalid rows and continues (unless `--strict`)
- Rejects an `asset_id` that already exists in the database or appears twice in the file
- Prints rows read / accepted / rejected and the rejects file path
- Appends one dated line to `logs/ingest.log`

Re-running the same CSV against a database that already holds those assets accepts **0** rows and rejects duplicates with `asset_id: is already in use`.

6. Cleaning & validation

Automatic cleaning

- Descriptions: trim, collapse spaces, consistent capitalisation
- Coordinates with compass letters (e.g. `28.6148 N`) → signed decimals
- Asset types matched regardless of letter case
- Surveyor names standardised so variants of one person count once
- `attribute_json` parsed into ordinary JSON data

Validation rules (CSV and API)

Field	Rule	On failure
asset_id	Present, <code>XX-1234</code> , not already in use	Reject
name	3–120 characters after tidying	Reject
asset_type	<code>pole, valve, manhole, transformer</code>	Reject
latitude	Number between <code>-90</code> and <code>90</code>	Reject
longitude	Number between <code>-180</code> and <code>180</code>	Reject
elevation_m	Number if present; blank → not recorded	Store null / reject if non-numeric
surveyed_on	Valid date, not in the future	Reject
status	<code>active, decommissioned, proposed</code>	Reject

Field	Rule	On failure
condition_score	Whole number 0–10	Reject
attribute_json	Valid JSON object	Reject
Cross-field	Decommissioned assets may not have condition > 2	Reject

Condition bands

Score	Band
8–10	GOOD
5–7	FAIR
3–4	POOR
0–2	CRITICAL

7. Web API

Start the service:

```
python main.py
```

- Interactive OpenAPI docs: <http://127.0.0.1:8000/docs>
- Liveness (no login): <http://127.0.0.1:8000/status>

Authentication

	Method	Path	Who	Purpose
POST	POST	/auth/login	Anyone	Sign in and receive a credential
POST	POST	/auth/users	Admin	Create a user account
GET	GET	/auth/me	Signed-in	Current user profile

Assets (six distinct operations)

	Path	Who	Purpose
GET	/assets	Signed-in	List assets (paged)
GET	/assets/{code}	Signed-in	Fetch one asset
POST	/assets	Signed-in	Add a new asset
PUT	/assets/{code}	Signed-in	Replace an asset in full
PATCH	/assets/{code}	Signed-in	Correct selected fields
DELETE	/assets/{code}	Admin	Remove asset and visit history

	Path	Who	Purpose
GET	/assets/{code}/visits	Signed-in	Visit history

List query parameters

Parameter	Description
type	Filter by asset type
status	Filter by status
surveyor	Filter by surveyor name
condition_min / condition_max	Condition score range
q	Case-insensitive word in the description
limit	Page size (default 25, maximum 100)
offset	Starting position

The list response always includes `items`, `total`, `limit`, and `offset`. A request never returns every asset at once.

Bulk upload

	Path	Who	Purpose
POST	/ingest/upload	Admin	Upload a day's CSV over the network; returns accepted / rejected counts

Error outcomes

Responses use a consistent body shape:

```
{
  "error": "validation_error | not_found | not_authenticated | forbidden | rate_limited",
  "message": "Readable explanation",
  "details": [{ "field": "...", "reason": "..." }],
  "duration_ms": 12.3
}
```

8. Reports

	Path	Purpose
GET	/reports/summary	Counts, averages, worst asset per type, survey extent, totals (cached up to 60s)
GET	/reports/repairs	In-service assets with condition < 5
GET	/reports/most-visited	Assets visited most often

	Path	Purpose
GET	/reports/nearest	Nearest asset to a lat/lon (great-circle km)
GET	/reports/surveyors	Distinct surveyors on a given day (?on=YYYY-MM-DD)
GET	/status	Public liveness check for monitoring

The summary cache is discarded immediately when any asset is added, corrected, or removed, so a stale summary is never served after a change.

9. Security & non-functional behaviour

- Passwords stored with bcrypt (not recoverable)
- JWT credentials expire after `JWT_EXPIRE_MINUTES`
- Secrets come from the environment — never hard-coded
- CORS limited to origins listed in `CORS_ORIGINS`
- Rate limit: 60 requests per minute per caller (`/status` exempt)
- Every request is logged with path, status, and duration
- JSON responses include `duration_ms` where applicable

10. Database

Table	Purpose
<code>users</code>	Accounts, hashed passwords, roles
<code>assets</code>	Current asset master data; unique <code>asset_id</code>
<code>visits</code>	Visit history (date, surveyor, condition, notes)

Deleting an asset cascades to its visits so no orphaned history remains. Switching from local PostgreSQL to an enterprise server is a change of `DATABASE_URL` only.

```
alembic upgrade head
```

11. Output files

File	Contents
<code>outputs/rejects.csv</code>	Every refused row plus a <code>reason</code> column
<code>outputs/assets.geojson</code>	Accepted assets as a GeoJSON FeatureCollection
<code>outputs/summary.txt</code>	Printable summary by type, extent, and totals

File	Contents
logs/ingest.log	Append-only dated ingest run log
logs/requests.log	HTTP request log
logs/app.log	Application log

Sample handheld export: data/survey_export.csv (62 rows; about 1 in 6 faulty).

12. Testing

```
pytest
```

Tests use a throwaway database named `asset_registry_test` and never touch live `asset_registry` data. Coverage includes cleaning boundaries, rejection of bad rows, duplicate re-ingest refusal, network CRUD, missing records, successful sign-in, and surveyor delete refusal.

13. Project layout

main.py	Start the API
requirements.txt	Pinned dependencies
.env / .env.example	Configuration (secrets not committed)
scripts/	
setup_local_database.py	Create local role + databases
init_local_database.sql	Same steps for pgAdmin
docs/	
documentation.html	This documentation
src/asset_registry/	
config.py	Settings from project-root .env
ingest/	CLI tool and CSV pipeline
domain/	Cleaning, validation, distance
db/	SQLAlchemy models and session
api/	FastAPI app, middleware, routes
auth/	Passwords, JWT, roles
services/	Assets, reports, summary cache
data/	Sample survey export
outputs/	Rejects, map, summary from a run
tests/	Automated tests

14. Quick reference

Start API

```
python main.py
```

Ingest CSV

```
python -m asset_registry.ingest data/survey_export.csv
```

Admin login

admin / Admin@123

Surveyor login

surveyor / Surveyor@123

API docs

<http://127.0.0.1:8000/docs>

Status

<http://127.0.0.1:8000/status>